



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Networking Management from The CLI

PowerShell has expanded our capabilities within the [Windows OS](#) when dealing with Networking settings, applications, and more. This section will cover how to check your network settings, such as IP addresses, adapter settings, and DNS settings. We will also cover How to enable and manage remote host access utilizing [WinRM](#) and [SSH](#).

Scenario: To ensure Mr. Tanaka's host is functioning properly and we can manage it from the IT office remotely, we are going to perform a quick checkup, validate his host settings, and enable remote management for the host.

What Is Networking Within a Windows Network?

Networking with Windows hosts functions much like any other Linux or Unix-based host. The TCP/IP stack, wireless protocols, and other applications treat most devices the same, so there isn't much to learn there that's new. This module assumes you know basic networking protocols and how typical network traffic traverses the Internet. If you wish for a primer on networking, check out the [Introduction to Networking](#) module, or for a more in-depth dissection of network traffic, you can play through the [Introduction to Network Traffic Analysis](#) module. Where things get a bit different lies in how Windows hosts communicate with each other, domains, and other Linux hosts. Below we will quickly cover some standard protocols you could run into when administering or pentesting Windows hosts.

Protocol	Description
SMB	SMB provides Windows hosts with the capability to share resources, files, and a standard way of authenticating between hosts to determine if access to resources is allowed. For other distros, SAMBA is the open-source option.
Netbios	NetBios itself isn't directly a service or protocol but a connection and conversation mechanism widely used in networks. It was the original transport mechanism for SMB , but that has since changed. Now it serves as an alternate identification mechanism when DNS fails. Can also be known as NBT-NS (NetBIOS name service).
LDAP	LDAP is an open-source cross-platform protocol used for authentication and authorization with various directory services. This is how many different devices in modern networks can communicate with large directory structure services such as Active Directory .
LLMNR	LLMNR provides a name resolution service based on DNS and works if DNS is not available or functioning. This protocol is a multicast protocol and, as such, works only on local links (within a normal broadcast domain, not across layer three links).
DNS	DNS is a common naming standard used across the Internet and in most modern network types. DNS allows us to reference hosts by a unique name instead of their IP address. This is how we can reference a website by "WWW.google.com" instead of "8.8.8.8". Internally this is how we request resources and access from a network.
HTTP/HTTPS	HTTP/S HTTP and HTTPS are the insecure and secure way we request and utilize resources over the Internet. These protocols are used to access and utilize resources such as web servers, send and receive data from remote sources, and much more.
Kerberos	Kerberos is a network level authentication protocol. In modern times, we are most likely to see it when dealing with Active Directory authentication when clients request tickets for authorization to use domain resources.
WinRM	WinRM Is an implementation of the WS-Management protocol. It can be used to manage the hardware and software functionalities of hosts. It is mainly used in IT administration but can also be used for host enumeration and as a scripting engine.
RDP	RDP is a Windows implementation of a network UI services protocol that provides users with a Graphical interface to access hosts over a network connection. This allows for full UI use to include the passing of keyboard and mouse input to the remote host.
SSH	SSH is a secure protocol that can be used for secure host access, transfer of files, and general communication between network hosts. It



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

Local vs. Remote Access?

Local Access

Local host access is when we are directly at the terminal utilizing its resources as you are right now from your PC. Usually, this will not require us to use any specific access protocols except when we request resources from networked hosts or attempt to access the Internet. Below we will showcase some cmdlets and other ways to check and validate network settings on our hosts.

Querying Networking Settings

Before doing anything else, let's validate the network settings on Mr. Tanaka's host. We will start by running the `IPConfig` command. This isn't a PowerShell native command, but it is compatible.

IPConfig

```
Networking Management from The CLI

PS C:\htb> ipconfig

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : .htb
    Link-local IPv6 Address . . . . . : fe80::c5ca:594d:759d:e0c1%11
    IPv4 Address. . . . . : 10.129.203.105
    Subnet Mask . . . . . : 255.255.0.0
    Default Gateway . . . . . : fe80::250:56ff:feb9:b9fc%11
                                10.129.0.1
```

As we can see, `ipconfig` will show us the basic settings of your network interface. We have as output the IPv4/6 addresses, our gateway, subnet masks, and DNS suffix if one is set. We can output the full network settings by appending the `/all` modifier to the `ipconfig` command like so:

```
Networking Management from The CLI

PS C:\htb> ipconfig /all

Windows IP Configuration

    Host Name . . . . . : ICL-WIN11
    Primary Dns Suffix . . . . . : greenhorn.corp
    Node Type . . . . . : Hybrid
    IP Routing Enabled. . . . . : No
    WINS Proxy Enabled. . . . . : No
    DNS Suffix Search List. . . . . : greenhorn.corp
                                    htb

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : .htb
    Description . . . . . : vmxnet3 Ethernet Adapter
    Physical Address. . . . . : 00-50-56-B9-4F-CB
    DHCP Enabled. . . . . : Yes
    Autoconfiguration Enabled . . . . : Yes
    IPv4 Address. . . . . : 10.129.203.105
    Subnet Mask . . . . . : 255.255.0.0
    Default Gateway . . . . . : 10.129.0.1
```



To keep your account secure, move your 2FA to HTB Account by June 27th

The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
Subnet Mask . . . . . : 255.255.0.0
Lease Obtained. . . . . : Monday, October 17, 2022 9:40:13 AM
Lease Expires . . . . . : Tuesday, October 25, 2022 10:10:16 AM
Default Gateway . . . . . : fe80::250:56ff:feb9:b9fc%11
                          10.129.0.1
DHCP Server . . . . . : 10.129.0.1
DHCPv6 IAID . . . . . : 335564886
DHCPv6 Client DUID. . . . . : 00-01-00-01-2A-3D-00-D6-00-50-56-B9-4F-CB
DNS Servers . . . . . : 1.1.1.1
                          8.8.8.8
NetBIOS over Tcpip. . . . . : Enabled
Connection-specific DNS Suffix Search List :
                          htb
```

Ethernet adapter Ethernet2:

```
Connection-specific DNS Suffix . :
Description . . . . . : vmxnet3 Ethernet Adapter #2
Physical Address. . . . . : 00-50-56-B9-F5-7E
DHCP Enabled. . . . . : No
Autoconfiguration Enabled . . . . : Yes
Link-Local IPv6 Address . . . . . : fe80::d1fb:79d5:6d0b:41de%14(Preferred)
IPv4 Address. . . . . : 172.16.5.100(Preferred)
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 172.16.5.1
DHCPv6 IAID . . . . . : 318787670
DHCPv6 Client DUID. . . . . : 00-01-00-01-2A-3D-00-D6-00-50-56-B9-4F-CB
DNS Servers . . . . . : 172.16.5.155
NetBIOS over Tcpip. . . . . : Enabled
```

Now we can see much more information than before. We are presented with output containing multiple adapters, [Host settings](#), more details about if our IP addresses were [manually](#) set or [DHCP leases](#), how long those leases are, and more. So, it appears Mr. Tanaka's host has a proper IP address configuration. Of note, and particularly interesting to us as pentesters, is that this host is dual-homed. We mean it has multiple network interfaces connected to separate networks. This makes Mr. Tanaka's host a great target if we are looking for a foothold in the network and wish to have a way to migrate between networks.

Let's look at [ARP](#) settings and see if his host has communicated with others on the network. As a refresher, ARP is a protocol utilized to [translate IP addresses to Physical addresses](#). The physical address is used at lower levels of the [OSI/TCP-IP](#) models for communication. To have it display the host's current ARP entries, we will use the `-a` switch.

ARP

Networking Management from The CLI

```
PS C:\htb> arp -a
```

```
Interface: 10.129.203.105 --- 0xb
```

Internet Address	Physical Address	Type
10.129.0.1	00-50-56-b9-b9-fc	dynamic
10.129.204.58	00-50-56-b9-5f-41	dynamic
10.129.255.255	ff-ff-ff-ff-ff-ff	static
224.0.0.22	01-00-5e-00-00-16	static
224.0.0.251	01-00-5e-00-00-fb	static
224.0.0.252	01-00-5e-00-00-fc	static
239.255.255.250	01-00-5e-7f-ff-fa	static
255.255.255.255	ff-ff-ff-ff-ff-ff	static

```
Interface: 172.16.5.100 --- 0xe
```

Internet Address	Physical Address	Type
172.16.5.155	00-50-56-b9-e2-30	dynamic
172.16.5.255	ff-ff-ff-ff-ff-ff	static



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

The output from `Arp -a` is pretty simple. We are provided with entries from our network adapters about the hosts it is aware of or has communicated with recently. Not surprisingly, since this host is fairly new, it has yet to communicate with too many hosts. Just the gateways, our remote host, and the host 172.16.5.155, the `Domain Controller` for `Greenhorn.corp`. Nothing crazy to be seen here. Now let's validate our DNS configuration is working properly. We will utilize `nslookup`, a built-in DNS querying tool, to attempt to resolve the IP address / DNS name of the Greenhorn domain controller.

Nslookup

Networking Management from The CLI

```
PS C:\htb> nslookup ACADEMY-ICL-DC

DNS request timed out.
    timeout was 2 seconds.
Server:  UnKnown
Address:  172.16.5.155

Name:     ACADEMY-ICL-DC.greenhorn.corp
Address:  172.16.5.155
```

Now that we have validated Mr. Tanakas DNS settings, let's check the open ports on the host. We can do so using `netstat -an`. Netstat will display current network connections to our host. The `-an` switch will print all connections and listening ports and place them in numerical form.

Netstat

Networking Management from The CLI

```
PS C:\htb> netstat -an

netstat -an

Active Connections

    Proto Local Address           Foreign Address         State
    TCP    0.0.0.0:22              0.0.0.0:0               LISTENING
    TCP    0.0.0.0:135             0.0.0.0:0               LISTENING
    TCP    0.0.0.0:445             0.0.0.0:0               LISTENING
    TCP    0.0.0.0:3389            0.0.0.0:0               LISTENING
    TCP    0.0.0.0:5040            0.0.0.0:0               LISTENING
    TCP    0.0.0.0:5985            0.0.0.0:0               LISTENING
    TCP    0.0.0.0:47001           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49664           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49665           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49666           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49667           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49668           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49671           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49673           0.0.0.0:0               LISTENING
    TCP    0.0.0.0:49674           0.0.0.0:0               LISTENING
    TCP    10.129.203.105:22       10.10.14.19:32557       ESTABLISHED
    TCP    172.16.5.100:139        0.0.0.0:0               LISTENING
    TCP    [::]:22                [::]:0                  LISTENING
    TCP    [::]:135                [::]:0                  LISTENING
    TCP    [::]:445                [::]:0                  LISTENING
    TCP    [::]:3389               [::]:0                  LISTENING
    TCP    [::]:5985               [::]:0                  LISTENING
    TCP    [::]:47001              [::]:0                  LISTENING
```



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
TCP    [::]:49671      [::]:0      LISTENING
TCP    [::]:49673      [::]:0      LISTENING
TCP    [::]:49674      [::]:0      LISTENING
UDP    0.0.0.0:123      *: *
<SNIP>
UDP    172.16.5.100:137 *: *
UDP    172.16.5.100:138 *: *
UDP    172.16.5.100:1900 *: *
UDP    172.16.5.100:54453 *: *
```

Now, you may need to gain a background in looking at network traffic or an understanding of standard ports and protocols, or else the above may look like gibberish. That's ok, though. Looking above, we can see what ports are open and if we have any active connections. From the output above, the ports open are all commonly used in Windows environments and would be expected. Most deal with Active Directory services and SSH. When looking at the connections, we see only one currently active session: our own SSH connection over TCP port 22.

Most of these commands we have practiced with up to this point are Windows built-in executables and are helpful for quick insight into a host, but not for much more. Below we will cover several cmdlets that are additions from PowerShell that allow us to manage our network connections granularly.

PowerShell Net Cmdlets

PowerShell has several powerful built-in cmdlets made to handle networking services and administration. The NetAdapter, NetConnection, and NetTCPIP modules are just a few that we will practice with today.

Net Cmdlets

Cmdlet	Description
<code>Get-NetIPInterface</code>	Retrieve all visible network adapter properties .
<code>Get-NetIPAddress</code>	Retrieves the IP configurations of each adapter. Similar to IPConfig .
<code>Get-NetNeighbor</code>	Retrieves the neighbor entries from the cache. Similar to arp -a .
<code>Get-Netroute</code>	Will print the current route table . Similar to IPRoute .
<code>Set-NetAdapter</code>	Set basic adapter properties at the Layer-2 level such as VLAN id, description, and MAC-Address.
<code>Set-NetIPInterface</code>	Modifies the settings of an interface to include DHCP status, MTU, and other metrics.
<code>New-NetIPAddress</code>	Creates and configures an IP address .
<code>Set-NetIPAddress</code>	Modifies the configuration of a network adapter.
<code>Disable-NetAdapter</code>	Used to disable network adapter interfaces.
<code>Enable-NetAdapter</code>	Used to turn network adapters back on and allow network connections.
<code>Restart-NetAdapter</code>	Used to restart an adapter. It can be useful to help push changes made to adapter settings .
<code>test-NetConnection</code>	Allows for diagnostic checks to be ran on a connection. It supports ping, tcp, route tracing, and more.

We aren't going to show each cmdlet in use, but it would be prudent to provide a quick reference for your use. First, we will start with Get-NetIPInterface.



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
PS C:\htb> get-netIPInterface
```

ifIndex	InterfaceAlias	AddressFamily	NLMtu(Bytes)	InterfaceMetric	Dhcp	ConnectionState	Port
20	Ethernet 3	IPv6	1500	25	Enabled	Disconnected	Ac
14	VMware Network Adapter VMnet8	IPv6	1500	35	Enabled	Connected	Ac
8	VMware Network Adapter VMnet2	IPv6	1500	35	Enabled	Connected	Ac
10	VMware Network Adapter VMnet1	IPv6	1500	35	Enabled	Connected	Ac
17	Local Area Connection* 2	IPv6	1500	25	Enabled	Disconnected	Ac
21	Bluetooth Network Connection	IPv6	1500	65	Disabled	Disconnected	Ac
15	Local Area Connection* 1	IPv6	1500	25	Disabled	Disconnected	Ac
25	Wi-Fi	IPv6	1500	40	Enabled	Connected	Ac
7	Local Area Connection	IPv6	1500	25	Enabled	Disconnected	Ac
1	Loopback Pseudo-Interface 1	IPv6	4294967295	75	Disabled	Connected	Ac
20	Ethernet 3	IPv4	1500	25	Enabled	Disconnected	Ac
14	VMware Network Adapter VMnet8	IPv4	1500	35	Disabled	Connected	Ac
8	VMware Network Adapter VMnet2	IPv4	1500	35	Disabled	Connected	Ac
10	VMware Network Adapter VMnet1	IPv4	1500	35	Disabled	Connected	Ac
17	Local Area Connection* 2	IPv4	1500	25	Disabled	Disconnected	Ac
21	Bluetooth Network Connection	IPv4	1500	65	Enabled	Disconnected	Ac
15	Local Area Connection* 1	IPv4	1500	25	Enabled	Disconnected	Ac
25	Wi-Fi	IPv4	1500	40	Enabled	Connected	Ac
7	Local Area Connection	IPv4	1500	1	Disabled	Disconnected	Ac
1	Loopback Pseudo-Interface 1	IPv4	4294967295	75	Disabled	Connected	Ac

This listing shows us our available interfaces on the host in a bit of a convoluted manner. We are provided plenty of metrics, but the adapters are broken up by [AddressFamily](#). So we see entries for each adapter twice if IPv4 and IPv6 are enabled on that particular interface. The [ifindex](#) and [InterfaceAlias](#) properties are particularly useful. These properties make it easy for us to use the other cmdlets provided by the [NetTCP/IP](#) module. Let's get the Adapter information for our Wi-Fi connection at [ifIndex](#) 25 utilizing the [Get-NetIPAddress](#) cmdlet.

Get-NetIPAddress

Networking Management from The CLI

```
PS C:\htb> Get-NetIPAddress -ifIndex 25
```

```
IPAddress      : fe80::a0fc:2e3d:c92a:48df%25
InterfaceIndex : 25
InterfaceAlias  : Wi-Fi
AddressFamily   : IPv6
Type            : Unicast
PrefixLength    : 64
PrefixOrigin    : WellKnown
SuffixOrigin    : Link
AddressState    : Preferred
ValidLifetime   : Infinite ([TimeSpan]::MaxValue)
PreferredLifetime : Infinite ([TimeSpan]::MaxValue)
SkipAsSource    : False
PolicyStore     : ActiveStore
```

```
IPAddress      : 192.168.86.211
InterfaceIndex  : 25
InterfaceAlias  : Wi-Fi
AddressFamily   : IPv4
Type            : Unicast
PrefixLength    : 24
PrefixOrigin    : Dhcp
SuffixOrigin    : Dhcp
AddressState    : Preferred
ValidLifetime   : 21:35:36
```




To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

This cmdlet has returned quite a bit of information as well. Notice how we used the `ifIndex` number to request the information? We can do the same with the `InterfaceAlias` as well. This cmdlet returns quite a bit of information, such as the index, alias, DHCP state, interface type, and other metrics. This mirrors most of what we would see if we issued the `IPconfig` executable from the command prompt. Now, what if we want to modify a setting on the interface? We can do so with the `Set-NetIPInterface` and `Set-NetIPAddress` cmdlets. In this example, let's say we want to change the DHCP status of the interface from `enabled`, to `disabled`, and change the IP from one automatically assigned by DHCP to one of our choosing manually set. We would accomplish this like so:

Set-NetIPInterface

```
Networking Management from The CLI

PS C:\htb> Set-NetIPInterface -InterfaceIndex 25 -Dhcp Disabled
```

By disabling the DHCP property with the `Set-NetIPInterface` cmdlet, we can now set our manual IP Address. We do that with the `Set-NetIPAddress` cmdlet.

Set-NetIPAddress

```
Networking Management from The CLI

PS C:\htb> Set-NetIPAddress -InterfaceIndex 25 -IPAddress 10.10.100.54 -PrefixLength 24

PS C:\htb> Get-NetIPAddress -ifindex 20 | ft InterfaceIndex,InterfaceAlias,IPAddress,PrefixLength

InterfaceIndex InterfaceAlias IPAddress PrefixLength
-----
20 Ethernet 3 fe80::7408:bbf:954a:6ae5%20 64
20 Ethernet 3 10.10.100.54 24

PS C:\htb> Get-NetIPInterface -ifindex 20 | ft ifIndex,InterfaceAlias,Dhcp

ifIndex InterfaceAlias Dhcp
-----
20 Ethernet 3 Disabled
20 Ethernet 3 Disabled
```

The above command now sets our IP address to `10.10.100.54` and the `PrefixLength` (also known as the subnet mask) to `24`. Looking at our checks, we can see that those settings are in place. To be safe, let's restart our network adapter and test our connection to see if it sticks.

Restart-NetAdapter

```
Networking Management from The CLI

PS C:\htb> Restart-NetAdapter -Name 'Ethernet 3'
```

As long as nothing goes wrong, you will not receive output. So when it comes to `Restart-NetAdapter`, no news is good news. The easiest way to tell the cmdlet which interface to restart is with the `Name` property, which is the same as the `InterfaceAlias` from previous commands we ran. Now, to ensure we still have a connection, we can use the `Test-NetConnection` cmdlet.

Test-NetConnection



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
ComputerName      : <snip>msedge.net
RemoteAddress     : 13.107.4.52
InterfaceAlias    : Ethernet 3
SourceAddress     : 10.10.100.54
PingSucceeded     : True
PingReplyDetails (RTT) : 44 ms
```

The Test-NetConnection is a powerful cmdlet, capable of testing beyond basic network connectivity to determine whether we can reach another host. It can tell us about our TCP results, detailed metrics, route diagnostics and more. It would be worthwhile to look at this article by Microsoft on [Test-NetConnection](#). Now that we have completed our task and validated Mr. Tanaka's network settings on his host, let's discuss remote access connectivity for a bit.

Remote Access

When we cannot access Windows systems or need to manage hosts remotely, we can utilize PowerShell, SSH, and RDP, among other tools, to perform our work. Let's cover the main ways we can enable and use remote access. First, we will discuss [SSH](#).

How to Enable Remote Access? (SSH, PSSessions, etc.)

Enabling SSH Access

We can use [SSH](#) to access [PowerShell](#) on a Windows system over the network. Starting in 2018, SSH via the [OpenSSH](#) client and server applications has been accessible and included in all Windows Server and Client versions. It makes for an easy-to-use and extensible communication mechanism for our administrative use. Setting up OpenSSH on our hosts is simple. Let's give it a try. We must install the SSH Server component and the client application to access a host via SSH remotely.

Setting up SSH on a Windows Target

We can set up an SSH server on a Windows target using the [Add-WindowsCapability](#) cmdlet and confirm that it is successfully installed using the [Get-WindowsCapability](#) cmdlet.

Networking Management from The CLI

```
PS C:\Users\htb-student> Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH*'

Name : OpenSSH.Client~~~~0.0.1.0
State : Installed

Name : OpenSSH.Server~~~~0.0.1.0
State : NotPresent

PS C:\Users\htb-student> Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0

Path      :
Online    : True
RestartNeeded : False

PS C:\Users\htb-student> Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH*'

Name : OpenSSH.Client~~~~0.0.1.0
State : Installed

Name : OpenSSH.Server~~~~0.0.1.0
State : NotPresent
```




To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
PowerShell - Verbose
```

```
PS C:\Users\htb-student> Get-WindowsCapability -Online | Where-Object Name -like 'OpenSSH*'
```

```
Name : OpenSSH.Client~~~~0.0.1.0
State : Installed
```

```
Name : OpenSSH.Server~~~~0.0.1.0
State : Installed
```

Starting the SSH Service & Setting Startup Type

Once we have confirmed SSH is installed, we can use the [Start-Service](#) cmdlet to start the SSH service. We can also use the [Set-Service](#) cmdlet to configure the startup settings of the SSH service if we choose.

Networking Management from The CLI

```
PS C:\Users\htb-student> Start-Service sshd
```

```
PS C:\Users\htb-student> Set-Service -Name sshd -StartupType 'Automatic'
```

Note: Initial setup of remote access services will not be a requirement in this module to complete challenge questions. With each of the challenges in this module, remote access is already set up & configured. However, understanding how to connect and apply concepts covered throughout the module will be required. The setup & configuration steps are provided to help develop an understanding of common configuration mistakes and, in some cases, best security practices. Feel free to try some setup steps on your own personal VM.

Accessing PowerShell over SSH

With SSH installed and running on a Windows target, we can connect over the network with an SSH client.

Connecting from Windows

Networking Management from The CLI

```
PS C:\Users\administrator> ssh htb-student@10.129.224.248
```

```
htb-student@10.129.224.248 password:
```

By default, this will connect us to a CMD session, but we can type `powershell` to enter a PowerShell session, as mentioned earlier in this section.

Networking Management from The CLI

```
WS01\htb-student@WS01 C:\Users\htb-student> powershell
```

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
PS C:\Users\htb-student>
```

We will notice that the steps to connect to a Windows target over SSH using Linux are identical to those when connecting from Windows.



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

```
PS C:\Users\administrator> ssh htb-student@10.129.224.248

htb-student@10.129.224.248 password:

WS01\htb-student@WS01 C:\Users\htb-student> powershell

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\htb-student>
```

Now that we have covered SSH let's spend some time covering enabling and using [WinRM](#) for remote access and management.

Enabling WinRM

[Windows Remote Management \(WinRM\)](#) can be configured using dedicated PowerShell cmdlets and we can enter into a PowerShell interactive session as well as issue commands on remote Windows target(s). We will notice that WinRM is more commonly enabled on Windows Server operating systems, so IT admins can perform tasks on one or multiple hosts. It's enabled by default in Windows Server.

Because of the increasing demand for the ability to remotely manage and automate tasks on Windows systems, we will likely see WinRM enabled on more & more Windows desktop operating systems (Windows 10 & Windows 11) as well. When WinRM is enabled on a Windows target, it listens on logical ports [5985](#) & [5986](#).

Enabling & Configuring WinRM

WinRM can be enabled on a Windows target using the following commands:

Networking Management from The CLI

```
PS C:\WINDOWS\system32> winrm quickconfig

WinRM service is already running on this machine.
WinRM is not set up to allow remote access to this machine for management.
The following changes must be made:

Enable the WinRM firewall exception.
Configure LocalAccountTokenFilterPolicy to grant administrative rights remotely to local users.

Make these changes [y/n]? y

WinRM has been updated for remote management.

WinRM firewall exception enabled.
Configured LocalAccountTokenFilterPolicy to grant administrative rights remotely to local users.
```

As can be seen in the above output, running this command will automatically ensure all the necessary configurations are in place to:

- Enable the WinRM service
- Allow WinRM through the Windows Defender Firewall (Inbound and Outbound)
- Grant administrative rights remotely to local users

As long as credentials to access the system are known, anyone who can reach the target over the network can connect after that command is run. IT admins should take further steps to harden these WinRM configurations, especially if the system will be remotely



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

- Join Windows systems to an Active Directory Domain Environment and Enforce Kerberos Authentication

Testing PowerShell Remote Access

Once we have enabled and configured WinRM, we can test remote access using the [Test-WSMan](#) PowerShell cmdlet.

Testing Unauthenticated Access

```
Networking Management from The CLI

PS C:\Users\administrator> Test-WSMan -ComputerName "10.129.224.248"

wsmid           : http://schemas.dmtf.org/wbem/wsman/identity/1/wsmanidentity.xsd
ProtocolVersion : http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd
ProductVendor   : Microsoft Corporation
ProductVersion  : OS: 0.0.0 SP: 0.0 Stack: 3.0
```

Running this cmdlet sends a request that checks if the WinRM service is running. Keep in mind that this is unauthenticated, so no credentials are used, which is why no **OS** version is detected. This shows us that the WinRM service is running on the target.

Testing Authenticated Access

```
Networking Management from The CLI

PS C:\Users\administrator> Test-WSMan -ComputerName "10.129.224.248" -Authentication Negotiate

wsmid           : http://schemas.dmtf.org/wbem/wsman/identity/1/wsmanidentity.xsd
ProtocolVersion : http://schemas.dmtf.org/wbem/wsman/1/wsman.xsd
ProductVendor   : Microsoft Corporation
ProductVersion  : OS: 10.0.17763 SP: 0.0 Stack: 3.0
```

We can run the same command with the option **-Authentication Negotiate** to test if WinRM is authenticated, and we will receive the OS version (**10.0.11764**).

PowerShell Remote Sessions

We also have the option to use the [Enter-PSSession](#) cmdlet to establish a PowerShell session with a Windows target.

Establishing a PowerShell Session

```
Networking Management from The CLI

PS C:\Users\administrator> Enter-PSSession -ComputerName 10.129.224.248 -Credential htb-student -Authentication Negotiate
[10.129.5.129]: PS C:\Users\htb-student\Documents> $PSVersionTable

Name                           Value
----                           -
PSVersion                      5.1.17763.592
PSEdition                      Desktop
PSCompatibleVersions           {1.0, 2.0, 3.0, 4.0...}
BuildVersion                   10.0.17763.592
CLRVersion                     4.0.30319.42000
WSManStackVersion              3.0
PSRemotingProtocolVersion      2.3
SerializationVersion           1.1.0.1
```



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

Enable now

Using Enter-PSSession from Linux

Networking Management from The CLI

```
dbcyph0n@htb[/htb]$ [PS]> Enter-PSSession -ComputerName 10.129.224.248 -Credential htb-student -Authentication  
PowerShell credential request  
Enter your credentials.  
Password for user htb-student: *****  
  
[10.129.224.248]: PS C:\Users\htb-student\Documents> $PSVersionTable  
  
Name                           Value  
----                           -  
PSVersion                       5.1.19041.1  
PSEdition                       Desktop  
PSCompatibleVersions             {1.0, 2.0, 3.0, 4.0...}  
BuildVersion                    10.0.19041.1  
CLRVersion                      4.0.30319.42000  
WSManStackVersion               3.0  
PSRemotingProtocolVersion       2.3  
SerializationVersion            1.1.0.1
```

Along with being OS agnostic, there are now tons of different tools that we can use to interact remotely with hosts. Picking a means to remotely administer our hosts mostly comes down to what you are comfortable with and what you can use based on the engagement or your environment security settings.

Networking is a pretty straightforward task to manage on Windows hosts. As your environments get more complex with cloud servers, multiple domains, and multiple sites across large geographical distances, network management at the level can get tedious. Luckily, we are only focused on our local host and how to manage a singular host. Moving forward, we will look at how we can interact with the web using PowerShell.

VPN Servers

Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

PROTOCOL

UDP 1337 TCP 443

DOWNLOAD VPN CONNECTION FILE



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

Enable now

∞ / 1 spawns left

Waiting to start...



Questions

Answer the question(s) below to complete this Section and earn cubes!

Target(s): Fetching status...



Cheat Sheet



Download VPN
Connection File

+ 0

What common protocol is used to resolve names to IP addresses.

Submit your answer here...

+10 Streak pts



Submit



Hint



Show Solution

+ 0

What PowerShell cmdlet will show us the IP configurations of the hosts network adapters.

Submit your answer here...

+10 Streak pts



Submit



Hint



Show Solution

+ 0

What command can enable and configure Windows Remote Management on a host?

Submit your answer here...

+10 Streak pts



Submit



Hint



Show Solution

← Previous

Next →



Cheat Sheet



Resources



Go to Questions



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)

CMD

 Command Prompt Basics	
 Getting Help	
 System Navigation	
 Working with Directories and Files	
 Gathering System Information	
 Finding Files and Directories	
 Environment Variables	
 Managing Services	
 Working With Scheduled Tasks	

PowerShell

 CMD Vs. PowerShell	
 All About Cmdlets and Modules	
 User and Group Management	
 Working with Files and Directories	
 Finding & Filtering Content	
 Working with Services	
 Working with the Registry	
 Working with the Windows Event Log	
 Networking Management from The CLI	
 Interacting with The Web	
 PowerShell Scripting and Automation	

Finish Strong

-  Skills Assessment

Beyond This Module

My Workstation

OFFLINE

 Start Instance

 / 1 spawns left



To keep your account secure, move your 2FA to HTB Account by June 27th
The 2FA of Academy, will be disabled soon, so be sure you have 2FA enabled on SSO side.

[Enable now](#)